GUÍA PRÁCTICA LAB 03 Y PROMPTS OPTIMIZADOS • EDICIÓN RETAIL OMNICANAL (RETAIL PULSE)

Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

TRYOLABS • GOOGLE ANTIGRAVITY WORKSHOP • LAB 03 (ES)

Laboratorio 03: Servidor MCP de Retail, Hooks Deterministas de Margen y Gobernanza de Seguridad Empresarial

Perfil: Track Técnico y Comercial | **Prerrequisitos**: Laboratorio 02 Completado

Idioma / Language: Español | English • **Guía de Prompts:** PROMPTS.es.md | PROMPTS.md • **PDF:** ES (Oscuro) | ES (Claro) | EN (Dark) | EN (Light)

1. Escenario de Retail y Contexto de Negocio

Incluso después de codificar reglas y skills (Laboratorio 02), persisten dos grandes desafíos operativos en una organización empresarial de retail:

1. **Conexión en Vivo a Sistemas mediante Model Context Protocol (MCP)**:

Ni los ingenieros ni los gerentes comerciales de categoría desean copiar y pegar consultas SQL o archivos CSV en ventanas de chat. En su lugar, Antigravity debe consultar directamente la base de datos en vivo **`data/retail_pulse.db`** mediante herramientas estructuradas de **Model Context Protocol (MCP)** para inspeccionar el True ATP por nodo, auditar contratos OTIF de proveedores en incumplimiento, simular márgenes promocionales antes de su lanzamiento y listar pedidos VIP en riesgo de quiebre de stock.

2. **Guardrails Deterministas y Gobernanza Empresarial**:

Las instrucciones en **`.agents/rules/`** guían el razonamiento del LLM, pero el cumplimiento de misión crítica —como bloquear mutaciones SQL destructivas (**`DROP`**/**`TRUNCATE`**/**`DELETE`** sobre **`wms_inventory`**, **`omnichannel_orders`** o **`vendor_otif_contracts`**) o impedir que alguien codifique **`margin_floor_pct = 0.0`** / **`DISABLE_MARGIN_FLOOR`**— requiere **ejecución determinista** fuera de la salida probabilística del LLM, junto con **gobernanza en Google Cloud / Gemini Enterprise** y **controles de Sandbox y Permisos del cliente Antigravity**.

2. Objetivos de Aprendizaje

En este laboratorio lograrás:

1. Completar el **Servidor FastMCP de Retail Pulse** (**`mcp_server/retail_mcp.py`**) implementando tres herramientas MCP de alto impacto junto a la herramienta inicial **`query_true_atp_inventory`**:

- **`audit_vendor_otif_contracts(only_breaching: bool = True)`**: Audita los contratos On-Time In-Full (OTIF) de proveedores y calcula penalizaciones deterministas por chargeback (**`$98,250.00`** en 6 proveedores en incumplimiento).

- **`simulate_promo_margin_guard(sku: str, qty: int = 1, promo_codes_csv: str = "")`**: Simula la acumulación de códigos promocionales sobre un SKU e informa si los descuentos acumulados activan el límite del piso de margen bruto del 15% (**`margin_floor_clamped`**).
- **`list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND")`**: Consulta **`omnichannel_orders`** + **`wms_inventory`** + **`sku_catalog`** y evalúa **`evaluate_location_atp(..., required_qty=row["qty"])["oversell_risk"]`** para listar los pedidos VIP en riesgo de quiebre de stock y su Customer Lifetime Value total en riesgo (**`total_vip_ltv_at_risk_usd`**).

2. Registrar el servidor MCP local en **`.agents/mcp_config.json`** para que Antigravity descubra e invoque automáticamente las 4 herramientas MCP de retail (**`"command": "./.venv/bin/python", "args": ["mcp_server/retail_mcp.py"]`**).
3. Configurar hooks deterministas de ciclo de vida **`PreToolUse`** y **`PostToolUse`** en **`.agents/hooks.json`** respaldados por **`scripts/margin_guard_hook.py`** (**`--phase pre`** y **`--phase post`**) que interceptan SQL destructivo o evasiones del piso de margen (**`exit code 2`** con **`{"decision": "deny", ...}`**) y escriben un registro de auditoría inmutable en **`data/logs/hook_compliance_audit.jsonl`**.
4. Recorrer la arquitectura de **Gobernanza de Google Cloud & Gemini Enterprise** y los **Controles de Seguridad del Cliente y Workspace de Antigravity** para despliegues empresariales de retail.

3. Instrucciones Paso a Paso

Paso 1: Inicializar el Estado Inicial del Laboratorio 03

Ejecuta el instalador del Laboratorio 03 para asegurar que las soluciones de los Laboratorios 01 y 02 estén activas y cargar los archivos iniciales del Laboratorio 03:

```
./labs/lab-03-mcp-and-hooks/install.sh
```

Verifica que la suite de pruebas del Laboratorio 03 falle inicialmente en las herramientas MCP aún no implementadas (**`TODO_LAB_03`**) y los archivos **`.agents/mcp_config.json`** / **`.agents/hooks.json`**:

```
./labs/lab-03-mcp-and-hooks/verify.sh
```

Paso 2: Inspeccionar el Servidor MCP Inicial (`mcp_server/retail_mcp.py`)

Abre **`mcp_server/retail_mcp.py`**. Observa que una herramienta MCP ya está implementada:

- **`query_true_atp_inventory(sku: str, location_id: str | None = None, required_qty: int = 1) -> str`**

Sin embargo, tres herramientas son actualmente stubs **`TODO_LAB_03`**:

- **`audit_vendor_otif_contracts(only_breaching: bool = True) -> str`**
- **`simulate_promo_margin_guard(sku: str, qty: int = 1, promo_codes_csv: str = "") -> str`**
- **`list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND") -> str`**

Paso 3: Implementar las Herramientas MCP Restantes y Registrar `.agents/mcp_config.json`

Abre **`labs/lab-03-mcp-and-hooks/PROMPTS.es.md`** y utiliza los **Prompts de Implementación** (o **Edición en Línea `Cmd+I` / `Ctrl+I`**) para:

1. Implementar **`audit_vendor_otif_contracts(only_breaching: bool = True) -> str`** en **`mcp_server/retail_mcp.py`** delegando en **`audit_all_vendor_otif_contracts(only_breaching=only_breaching)`**.
2. Implementar **`simulate_promo_margin_guard(sku: str, qty: int = 1, promo_codes_csv: str = "") -> str`** en **`mcp_server/retail_mcp.py`** separando **`promo_codes_csv`** en una lista de códigos en mayúsculas y llamando a **`calculate_promotional_order_pricing(sku=sku, qty=qty, promo_codes=codes)`**.
3. Implementar **`list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND") -> str`** en **`mcp_server/retail_mcp.py`** uniendo **`omnichannel_orders`**, **`wms_inventory`** y **`sku_catalog`**, evaluando **`evaluate_location_atp(..., required_qty=int(row["qty"]))`**, filtrando los pedidos donde **`atp_eval["oversell_risk"]`** sea **`True`** (o **`status == "AT_RISK_STOCKOUT"`**), y devolviendo **`loyalty_tier_filter`**, **`at_risk_order_count`**, **`total_vip_ltv_at_risk_usd`** y **`orders`**.
4. Actualizar **`.agents/mcp_config.json`** registrando el servidor **`retail-pulse-mcp`** mediante **`"command": "./.venv/bin/python", "args": ["mcp_server/retail_mcp.py"]`**.

Paso 4: Configurar los Hooks Deterministas `PreToolUse` y `PostToolUse` de Protección de Margen (`.agents/hooks.json`)

Inspecciona **`scripts/margin_guard_hook.py`**. Este script lee el payload de la llamada a herramienta desde **`stdin`**:

- Durante **`--phase pre`** (**`PreToolUse`**), si un comando o edición de archivo intenta SQL destructivo (**`DROP TABLE`**, **`TRUNCATE`** o **`DELETE FROM wms_inventory|omnichannel_orders|vendor_otif_contracts`**) o intenta desactivar el piso de margen del 15% (**`margin_floor_pct = 0.0`** o **`DISABLE_MARGIN_FLOOR`**), el hook finaliza con código **`2`** y devuelve **`{"decision": "deny", "reason": "..."}`**.
- Cada evaluación del hook (**`allow`** o **`deny`**) se agrega como un evento JSON estructurado en **`data/logs/hook_compliance_audit.jsonl`**.

Configura **`.agents/hooks.json`** para conectar **`./.venv/bin/python scripts/margin_guard_hook.py --phase pre`** al evento **`PreToolUse`** (**`run_command|execute_command|bash|write_file|edit_file`**) y **`./.venv/bin/python scripts/margin_guard_hook.py --phase post`** a **`PostToolUse`**.

Prueba el hook directamente desde la terminal para verificar tanto el bloqueo como el registro en el log de auditoría:

```
echo '{"tool_name":"edit_file","tool_input":{"path":"src/services/promo_pricing.py","con \
  tent":"margin_floor_pct = 0.0 # DISABLE_MARGIN_FLOOR"}}' | ./.venv/bin/python scripts/ \
  margin_guard_hook.py --phase pre
cat data/logs/hook_compliance_audit.jsonl
```

4. Inmersión Profunda en Seguridad, Privacidad y Gobernanza Empresarial

Al desplegar Antigravity en una empresa de retail (merchandising, cadena de suministro, operaciones en tienda e ingeniería de plataforma), los arquitectos de seguridad gobiernan la plataforma en **dos capas complementarias**:

```
+-------------------------------------------------------------------------------+
| CAPA 1: Gobernanza en Google Cloud & Gemini Enterprise (Plano de Admin Cloud)  |
|  - IAM e Identidad: roles/cloudaicompanion.user vía Cloud Identity / SSO Corp  |
|  - Cloud Audit Logs: cloudaicompanion.googleapis.com en Google Cloud Logging   |
|  - Privacidad: Cero Retención de Datos y Sin Entrenamiento con prompts/código  |
|  - Perímetro de Red: VPC Service Controls (VPC-SC) y Endpoints Regionales      |
```

```
|  - Protección IP: Indemnización de Google Cloud y Filtros de Citación             |
+-----------------------------------------------------------------------------------+
                                      |
                                      v
+-----------------------------------------------------------------------------------+
| CAPA 2: Controles de Seguridad del Cliente y Workspace Antigravity (Plano Local)  |
|  - Política de Ejecución: strict | request-review | proceed-in-sandbox             |
|  - Sandbox de Terminal y Aislamiento de Red: Sandboxing a nivel de SO             |
|  - Acceso a Archivos Fuera del Workspace: Restricción estricta al repositorio     |
|  - Lista Blanca/Negra de Comandos y Lista Blanca del Navegador                    |
|  - Hooks Deterministas de Ciclo de Vida: .agents/hooks.json -> auditoría JSONL    |
+-----------------------------------------------------------------------------------+
```

Capa 1: Gobernanza en Google Cloud & Gemini Enterprise

Área de Control	Capacidad en Google Cloud / Gemini Enterprise	Caso de Uso en Retail Empresarial
IAM y Gestión de Licencias	Asignar **`roles/cloudaicompanion.user`** mediante Google Cloud IAM integrado con grupos SSO de **Cloud Identity / Okta / Azure AD (Entra ID)**.	Otorgar acceso diferenciado a **`retail-platform-eng@`** y **`retail-commercial-ops@`** con alta/baja centralizada.
Cloud Audit Logging	Habilitar logs de auditoría **Admin Activity** y **Data Access** para **`cloudaicompanion.googleapis.com`** en **Google Cloud Logging**, y enrutarlos mediante Log Sinks a BigQuery o Chronicle SIEM.	Mantener una pista de auditoría empresarial inmutable de quién invocó Gemini Enterprise, desde qué IP/dispositivo y cuándo.
Privacidad de Datos y Cero Entrenamiento	Los prompts de clientes, contexto del workspace, esquemas SQL y código generado **nunca se utilizan para entrenar modelos fundacionales compartidos** y cumplen garantías de cero retención fuera de tu tenant.	Analizar de forma segura contratos confidenciales de proveedores (**`data/exports/vendor_contracts.csv`**), costos unitarios mayoristas y LTV de clientes VIP.
VPC Service Controls (VPC-SC)	Ubicar **`cloudaicompanion.googleapis.com`** dentro de un **perímetro de VPC Service Controls** junto con BigQuery, Cloud SQL y Secret Manager, con fijación de endpoints regionales.	Prevenir la exfiltración de datos desde estaciones de trabajo corporativas o redes externas no autorizadas.

Capa 2: Controles de Seguridad del Cliente y Workspace de Antigravity

Configuración / Hook del Cliente	Opciones de Configuración	Configuración Recomendada en Producción Retail
Política de Ejecución de Herramientas	**`strict`**, **`request-review`**, **`proceed-in-sandbox`**, **`always-proceed`**	Usar **`proceed-in-sandbox`** o **`request-review`** para que los comandos mutantes requieran sandboxing del SO o aprobación humana.

Configuración / Hook del Cliente	Opciones de Configuración	Configuración Recomendada en Producción Retail
Sandbox de Terminal y Política de Red	Sandbox a nivel de SO (**`sandbox-exec`** en macOS / namespaces en Linux) + Interruptor de Acceso a Internet	Habilitar el Sandbox de Terminal y restringir llamadas de red externas para alcanzar solo endpoints internos aprobados.
Acceso a Archivos Fuera del Workspace	Bloquear o requerir confirmación para lecturas/escrituras fuera de la raíz del workspace	**Bloquear** el acceso fuera del workspace para impedir lecturas de **`~/.ssh`**, **`~/.aws`** u otros directorios del host.
Lista Blanca / Negra de Comandos	Lista blanca de prefijos (**`pytest`**, **`.venv/bin/python`**) y lista negra (**`rm -rf`**, **`curl`**, **`nc`**)	Permitir **`.venv/bin/python -m pytest`** para TDD autónomo bloqueando comandos destructivos o de exfiltración.
Lista Blanca del Navegador	Lista blanca de dominios para el subagente **`/browser`**	Restringir **`/browser`** estrictamente a **`http://localhost:8002`** (**`http://127.0.0.1:8002`**) y dashboards internos aprobados.
Hooks Deterministas de Ciclo de Vida	**`.agents/hooks.json`** (**`PreToolUse`**, **`PostToolUse`**)	Hacer cumplir invariantes comerciales y de seguridad (como **`scripts/margin_guard_hook.py`**) y registrar cada verificación en **`data/logs/hook_compliance_audit.jsonl`**.

5. Lista de Verificación

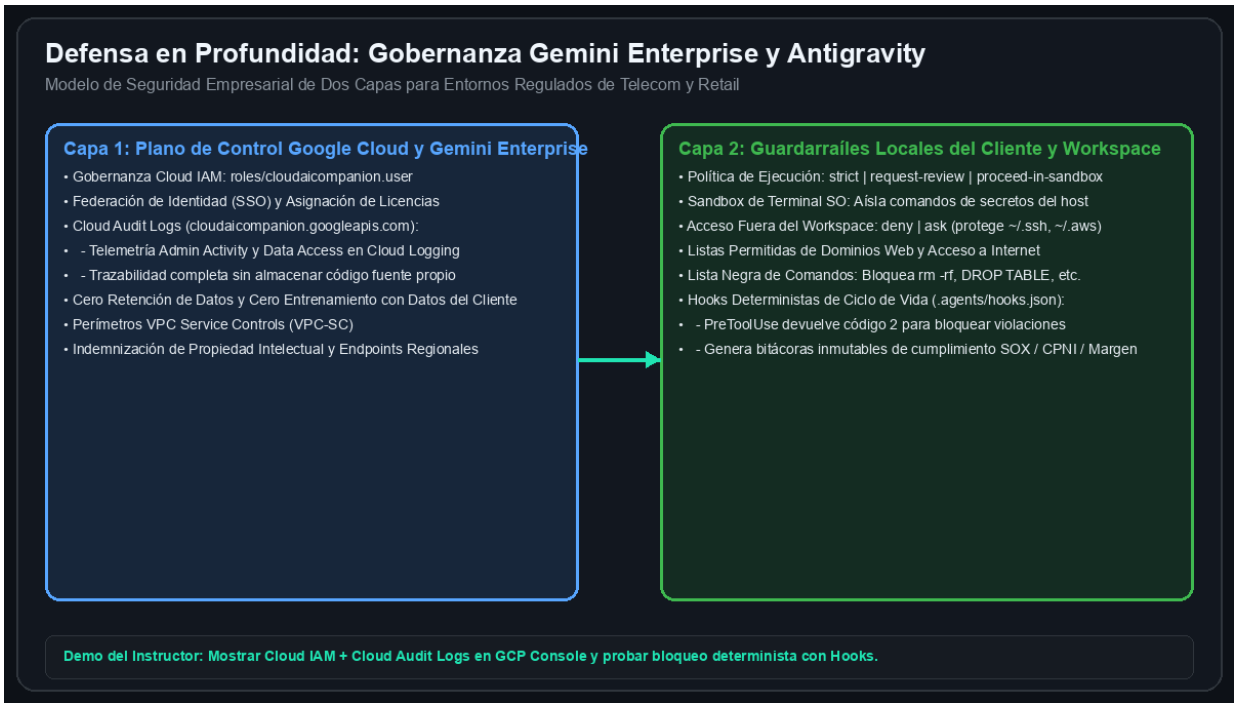
Ejecuta el script de verificación del Laboratorio 03 para confirmar que tus herramientas MCP, **`.agents/mcp_config.json`** y **`.agents/hooks.json`** pasan todas las pruebas automatizadas:

```
./labs/lab-03-mcp-and-hooks/verify.sh
```

Resultado esperado:

- **`tests/test_lab03_mcp_hooks.py`** pasa **3/3 pruebas del Laboratorio 03** (**13/13 pruebas acumuladas** entre los Laboratorios 01–03):
- **`test_mcp_config_and_hooks_json_are_valid`**
- **`test_retail_mcp_tools_return_accurate_live_sqlite_data`**
- **`test_margin_guard_hook_blocks_unsafe_mutations_and_allows_safe_commands`**

¿Necesitas saltar directamente al estado verificado? Ejecuta **`./labs/lab-03-mcp-and-hooks/apply_solution.sh`**.


Defensa en Profundidad: Gobernanza Gemini Enterprise y Antigravity
Modelo de Seguridad Empresarial de Dos Capas para Entornos Regulados de Telecom y Retail
Capa 1: Plano de Control Google Cloud y Gemini Enterprise
• Gobernanza Cloud IAM: roles/cloudaicompanion.user
• Federación de Identidad (SSO) y Asignación de Licencias
• Cloud Audit Logs (cloudaicompanion.googleapis.com):
• - Telemetría Admin Activity y Data Access en Cloud Logging
• - Trazabilidad completa sin almacenar código fuente propio
• Cero Retención de Datos y Cero Entrenamiento con Datos del Cliente
• Perímetros VPC Service Controls (VPC-SC)
• Indemnización de Propiedad Intelectual y Endpoints Regionales
Capa 2: Guardarraíles Locales del Cliente y Workspace
• Política de Ejecución: strict | request-review | proceed-in-sandbox
• Sandbox de Terminal SO: Aísla comandos de secretos del host
• Acceso Fuera del Workspace: deny | ask (protege ~/.ssh, ~/.aws)
• Listas Permitidas de Dominios Web y Acceso a Internet
• Lista Negra de Comandos: Bloquea rm -rf, DROP TABLE, etc.
• Hooks Deterministas de Ciclo de Vida (.agents/hooks.json):
• - PreToolUse devuelve código 2 para bloquear violaciones
• - Genera bitácoras inmutables de cumplimiento SOX / CPNI / Margen
Demo del Instructor: Mostrar Cloud IAM + Cloud Audit Logs en GCP Console y probar bloqueo determinista con Hooks.

LAB 03 PROMPTS OPTIMIZADOS PARA COPIAR Y PEGAR (ES)
Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

Playbook de Prompts del Laboratorio 03: Servidor MCP de Retail y Hooks Deterministas de Margen

Idioma / Language: Español | English • **Guía del Lab:** README.es.md | README.md • **PDF:** ES | EN

Copia y pega estos prompts en **Google Antigravity** a medida que avanzas en el Laboratorio 03. Cada sección incluye prompts diseñados tanto para el **Track Técnico** como para el **Track Comercial / Merchandising**.

Fase 1: Descubrimiento e Inspección de Arquitectura

Prompt 1A — Track Técnico: Inspeccionar el Servidor MCP Inicial y el Script del Hook

```
Lee @mcp_server/retail_mcp.py, @scripts/margin_guard_hook.py y @tests/test_lab03_mcp_hoo \
  ks.py.
Resume:
1. Cómo la herramienta MCP inicial (`query_true_atp_inventory`) se conecta a `data/retai \
  l_pulse.db` y a nuestra capa de servicios.
2. Qué campos JSON exactos deben devolver `audit_vendor_otif_contracts`, `simulate_promo \
  _margin_guard` y `list_at_risk_vip_orders` para pasar `tests/test_lab03_mcp_hooks.py`.
3. Cómo `scripts/margin_guard_hook.py` inspecciona los payloads `--phase pre` (`PreToolU \
  se`) y `--phase post` (`PostToolUse`) en stdin y escribe eventos de cumplimiento en `d \
  ata/logs/hook_compliance_audit.jsonl`.
```

Prompt 1B — Track Comercial / Operaciones: Comprender MCP en Vivo vs. Hooks Deterministas

```
Lee @mcp_server/retail_mcp.py y @scripts/margin_guard_hook.py. Explica en términos comer \
  ciales de retail:
1. Cómo las 4 herramientas MCP permiten a un gerente de Merchandising o Supply Chain con \
  sultar en vivo el stock True ATP, los contratos OTIF en incumplimiento, el impacto pro \
  mocional en márgenes y los pedidos VIP en riesgo sin escribir SQL.
2. Por qué combinamos nuestras directrices en `.agents/rules/retail-standards.md` con un \
   hook determinista `PreToolUse` (`scripts/margin_guard_hook.py`) que bloquea cualquier \
   intento de desactivar el piso de margen del 15% o ejecutar SQL destructivo sobre `wms \
  _inventory` y `omnichannel_orders`.
```

Fase 2: Planificación (`/plan`)

Prompt 2 — Crear un Plan de Implementación para Herramientas MCP y Hooks de Ciclo de Vida

```
/plan Diseña la implementación del Laboratorio 03 para que pasen todas las pruebas en @t \
  ests/test_lab03_mcp_hooks.py:
1. Implementar `audit_vendor_otif_contracts(only_breaching: bool = True) -> str`, `simul \
  ate_promo_margin_guard(sku: str, qty: int = 1, promo_codes_csv: str = "") -> str` y `l \
```

```
   ist_at_risk_vip_orders(loyalty_tier: str = "DIAMOND") -> str` en @mcp_server/retail_mc \
   p.py.
2. Actualizar `.agents/mcp_config.json` registrando `retail-pulse-mcp` usando `"command" \
   : "./.venv/bin/python"` y `"args": ["mcp_server/retail_mcp.py"]`.
3. Actualizar `.agents/hooks.json` registrando `./.venv/bin/python scripts/margin_guard_ \
   hook.py --phase pre` en `PreToolUse` (`run_command|execute_command|bash|write_file|edi \
   t_file`) y `./.venv/bin/python scripts/margin_guard_hook.py --phase post` en `PostTool \
   Use`.
```

Fase 3: Edición en Línea (`Cmd+I` / `Ctrl+I`)

Abre **`mcp_server/retail_mcp.py`**, selecciona las funciones stub **`audit_vendor_otif_contracts`**, **`simulate_promo_margin_guard`** y **`list_at_risk_vip_orders`**, presiona **`Cmd+I`** (macOS) o **`Ctrl+I`** (Windows/Linux) e ingresa:

```
Implementa `audit_vendor_otif_contracts` usando `audit_all_vendor_otif_contracts(only_br \
  eaching=only_breaching)`. Implementa `simulate_promo_margin_guard` separando `promo_co \
  des_csv` en una lista de códigos en mayúsculas y llamando a `calculate_promotional_ord \
  er_pricing(sku=sku, qty=qty, promo_codes=codes)`. Implementa `list_at_risk_vip_orders` \
   uniendo `omnichannel_orders`, `wms_inventory` y `sku_catalog`, evaluando `evaluate_lo \
  cation_atp` para cada pedido y devolviendo JSON con `loyalty_tier_filter`, `at_risk_or \
  der_count`, `total_vip_ltv_at_risk_usd` y `orders`.
```

Fase 4: Prompts de Implementación Completa

Prompt 4A — Implementar `mcp_server/retail_mcp.py`, `.agents/mcp_config.json` y `.agents/hooks.json`

```
Completa el Laboratorio 03 realizando los siguientes cambios:
1. En @mcp_server/retail_mcp.py:
   - Implementa `audit_vendor_otif_contracts(only_breaching: bool = True) -> str` devolv \
  iendo `json.dumps(audit_all_vendor_otif_contracts(only_breaching=only_breaching), inde \
  nt=2)`.
   - Implementa `simulate_promo_margin_guard(sku: str, qty: int = 1, promo_codes_csv: st \
  r = "") -> str` separando `promo_codes_csv` en códigos en mayúsculas y devolviendo `js \
  on.dumps(calculate_promotional_order_pricing(sku=sku, qty=qty, promo_codes=codes), ind \
  ent=2)`.
   - Implementa `list_at_risk_vip_orders(loyalty_tier: str = "DIAMOND") -> str` consulta \
  ndo `omnichannel_orders` unido con `wms_inventory` y `sku_catalog`, filtrando los pedi \
  dos donde `evaluate_location_atp(..., required_qty=int(row["qty"]))["oversell_risk"]`  \
  sea True (o `status == 'AT_RISK_STOCKOUT'`), y devolviendo JSON con `loyalty_tier_filt \
  er`, `at_risk_order_count`, `total_vip_ltv_at_risk_usd` y `orders`.
2. Actualiza `.agents/mcp_config.json` con una entrada `mcpServers` para `retail-pulse-m \
  cp` ejecutando `"command": "./.venv/bin/python"` y `"args": ["mcp_server/retail_mcp.py \
  "]`.
3. Actualiza `.agents/hooks.json` con `PreToolUse` (`./.venv/bin/python scripts/margin_g \
  uard_hook.py --phase pre`) y `PostToolUse` (`./.venv/bin/python scripts/margin_guard_h \
  ook.py --phase post`).
```

Fase 5: Verificación y Consultas MCP en Vivo

Prompt 5A — Ejecutar la Verificación Automatizada (`verify.sh`)

```
Ejecuta `./labs/lab-03-mcp-and-hooks/verify.sh`, confirma que las 13 pruebas acumuladas  \
  de los Labs 01-03 pasen (incluyendo las 3 pruebas en `tests/test_lab03_mcp_hooks.py`)  \
```

```
  y muestra la última entrada en `data/logs/hook_compliance_audit.jsonl`.
```

Prompt 5B — Track Comercial: Consultar las Herramientas MCP de Retail para Insights Ejecutivos

```
Usando las funciones en @mcp_server/retail_mcp.py, responde estas tres preguntas comerci \
  ales:
1. Si acumulamos las promociones `VIP20,FLASH15,CLEARANCE10` sobre `SKU-OLED-65` (qty=1) \
  , ¿se activa el límite del piso de margen del 15% (`margin_floor_clamped`) y cuál es e \
  l precio unitario ajustado?
2. ¿Cuántos pedidos VIP `DIAMOND` están actualmente en riesgo de quiebre de stock y cuál \
   es el Customer LTV total expuesto (`total_vip_ltv_at_risk_usd`)?
3. ¿Qué 6 proveedores en incumplimiento no alcanzan su SLA OTIF contratado y cuál es su  \
  penalización combinada por chargeback en USD (`total_chargeback_usd`)?
```

Fase 6: Desafío Adicional — Probar el Hook de Margen ante Prompts Adversarios

Prompt 6 — Intentar Evadir el Piso de Margen del 15% (Observar el Bloqueo del Hook)

```
Simula una solicitud no autorizada: prueba ejecutar `scripts/margin_guard_hook.py --phas \
  e pre` contra un payload que intente editar `src/services/promo_pricing.py` con `margi \
  n_floor_pct = 0.0 # DISABLE_MARGIN_FLOOR`, y otro payload que intente ejecutar `DROP T \
  ABLE wms_inventory;`. Muestra los códigos de salida (`2`), las decisiones JSON `deny`  \
  y las entradas registradas en `data/logs/hook_compliance_audit.jsonl`.
```